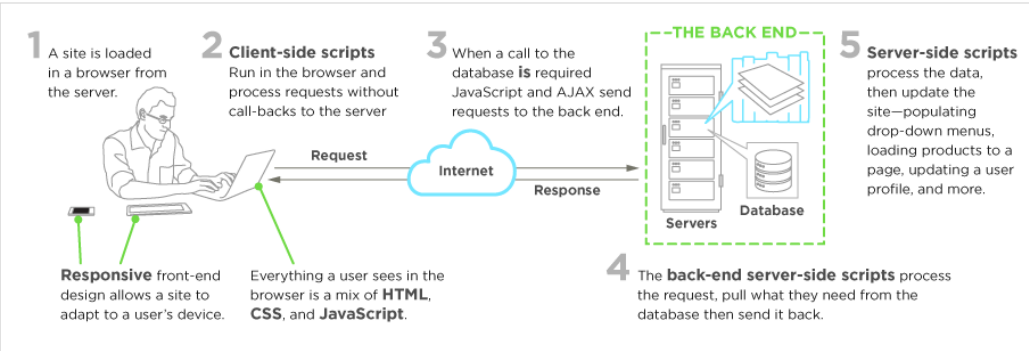
JavaScript

Instituto Superior de Engenharia
Politécnico de Coimbra

Linguagens de script

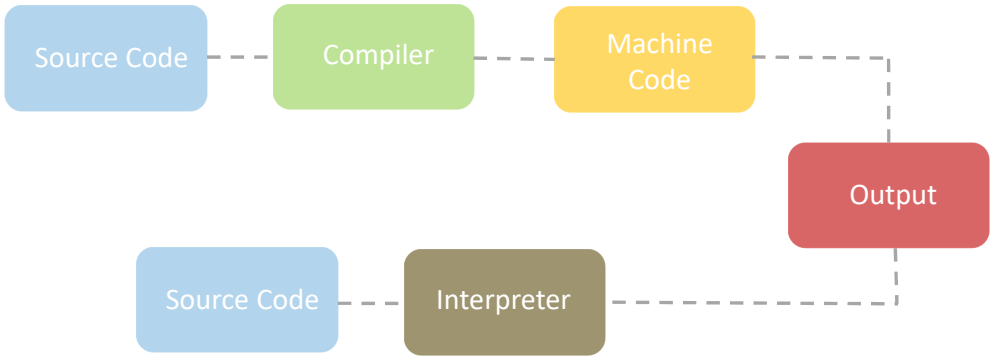
- Linguagem de programação integrada em outro programa/código
  - Linguagens interpretadas
  - Não necessitam de compilador
    - Javascript; PHP; …



https://www.upwork.com/hiring/development/how-scripting-languages-work/

Linguagens de script

- Linguagens Interpretada vs. Linguagens Compilada



Interpretação sequencial linha a linha
Deteção de erro por linha de código
Processo mais lento que uma linguagem compilada
Cross platform

JavaScript

- *Scripting language*

*"JavaScript is **THE** scripting language of the Web."*
http://www.w3schools.com/js/default.asp

  - Começou por ser exclusivamente uma ***client-side scripting language***
    - Interpretada diretamente pelo *browser (on the fly),* não necessita de ser compilada
  - Atualmente também utilizada no lado do servidor (*server-side*)
    - *Node.js*
  - Executado
    - Após o download
    - Como resposta a um evento
  - Permite:
    - Geração dinâmica de conteúdo / Efeitos
    - Melhorar a experiência do utilizador (interactividade)
      - Resposta a eventos, validação de dados, …
    - Gerir a comunicação com o servidor
    - ….



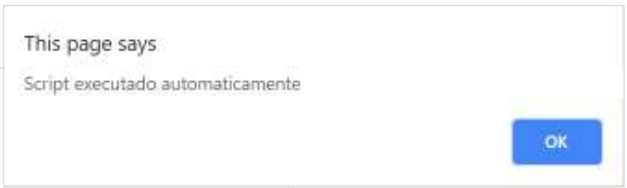
Inserção de scripts

Embedded Script

- *<script> … </script>*
  - O *script* pode ser colocado no *head* ou no *body*
    - preferencialmente, para maximizar a performance, o *script* deve ser colocado no final do body não influenciando assim o tratamento dos restantes elementos HTML
  - O *script* pode ser executado quando é efetuado o ***download*** do *.html (sem controlo por um evento)

```
<script>
     init();

     function init(){
         alert('Script executado automaticamente');
     }
 </script>
```



Embedded Script
(diretamente definido entre as tags <script>)

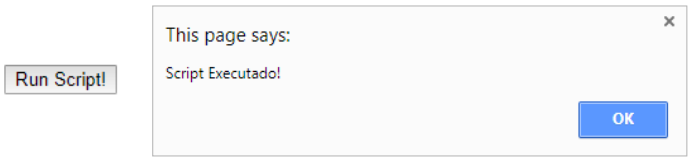
Embedded Script

- ***<script> … </script>***
  - o *script* pode ser executado como resposta a um evento, e.g. *onclick*

```
<button onclick="scriptFunction()">Run Script!</button>

<script>
    function scriptFunction(){
        alert('Script Executado!');
    }
</script>
```

Embedded Script
(diretamente definido entre as tags <script>)



Script Externo (*.js)

- ***<script src=" *.js "> … </script>***

```
<body>

    <button onclick="scriptFunction()">Run Script!</button>

    <script src="external.js"></script>

</body>
```

Executa a função scriptFunction() declarada no ficheiro external.js

Ligação ao **ficheiro externo** na tag **<script>** atributo **src**

```
function scriptFunction(){
        alert('Script Executado!');
  }
```

external.js

JavaScript

- Scripts

Embebido no HTML

Ficheiros Externos (extensão *.js)

- Controlo da execução do script

Executado após o download

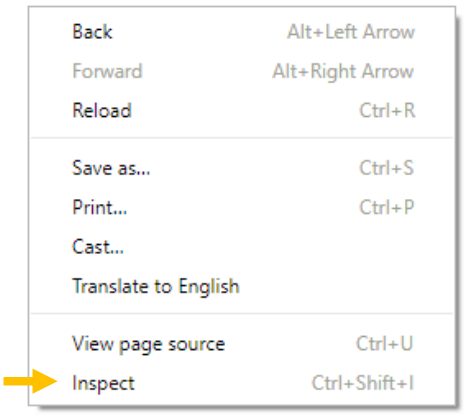
Executado só após a ocorrência de um evento

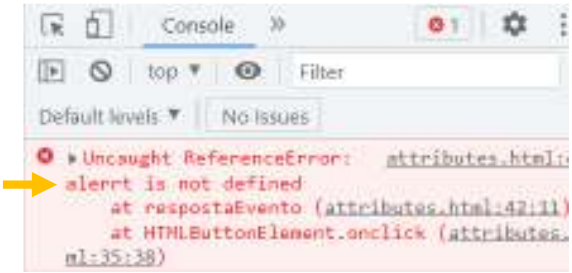
Chamada a uma função, em que a função pode ser:

- Criada pelo utilizador
- Nativa

Browser (development tools)

- *Browser Console*
  - Permite obter informação sobre o valor das variáveis, erros, *warnings*, *requests*, …
  - Importante para efetuar o *debugging*/controlo completo dos scripts
    - Linguagem interpretada (sequencial linha a linha)

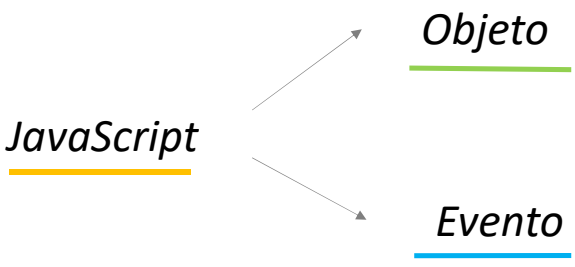




Conceitos Chave

JS

Conceitos Chave



Conceitos Chave

- Objeto
  - armazenar dados, estruturação da aplicação, código mais limpo/modular
    - Identidade
    - Propriedades
    - Métodos

```
<script>
    var hotel = {

        name: 'Coimbra',
        rooms: 20,
        booked: 15,
        gym: true,
        roomTypes:['single', 'double','suite'],

        checkAvailability: function () {
            return this.rooms - this.booked;
        }

    }
</script>
```



Javascript

- Evento
  - Ação que pode ser detetada pelo *JavaScript* e que provoca uma execução específica:
    - Chamada de uma função
    - A função **só é executada após a ocorrência** do respetivo evento
  - Exemplos:

Evento	É disparado...
click	quando é pressionado e liberado o botão primário do mouse, trackpad, etc.
mousemove	sempre que o cursor do mouse se move.
mouseover	quando o cursor do mouse é movido para sobre algum elemento.
mouseout	quando o cursor do mouse se move para fora dos limites de um elemento.
dblclick	quando acontece um clique duplo com o mouse, trackpad, etc.

http://desenvolvimentoparaweb.com/javascript/eventos-javascript/

Sintaxe

JS

Sintaxe JS

- *case sensitive*.
- // símbolo do comentário
  - /* comentário para múltiplas linhas */
- Um *script* é composto por um conjunto de *statements/expressions*

```
function Calculo(formulario)
{
    if(confirm("Confirma?"))
        formulario.result.value = eval(formulario.expr.value);
    else
        alert("Novos dados");
}
```

- Os *code blocks* (conjuntos de instruções) são delimitados por **{ … }**
  - Elementos fundamentais para a estruturação do código

Sintaxe JS

- *Expression*
  - a sua execução origina sempre um valor:
    - *numérico*
    - *string*
    - *boolean*
  - podem ser parte de *statements*

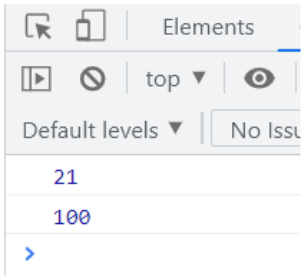
```
var sum;
var a=2;

 function Modify(i){
  return i*=10
 }

 sum = 20;    // assign 20
 sum++;       // increment

 a=Modify(10); // modifies the value of a

 console.log(sum);
 console.log(a);
```



Sintaxe JS

- *Statement*
  - a sua execução **produz uma ação mas não gera um valor imediato**
    - podem conter *expressions*
    - são executados isoladamente pela ordem em que são escritos

```
var sum;   // statement
var a=2    //  statement + assignment expression

 function Modify(i){        //function declaration statement
  return i*=10
 }

 sum = 20;
 sum++;

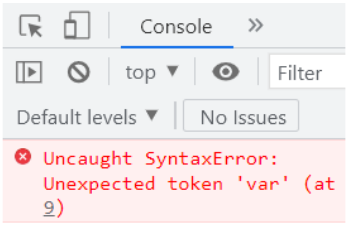
 a=Modify(10);

 console.log(sum);
 console.log(a);
```

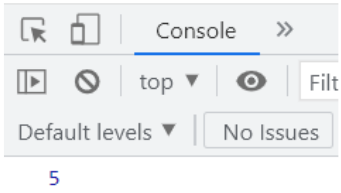
Sintaxe JS

- *Statements* não podem ser utilizados onde é esperada uma *expression*

```
var sum = var a
```




```
var a=2;
var b = (a=5);
console.log(b);
```



Sintaxe JS

- A utilização do **";"** não é consensual, existem 2 perspetivas:
  - *"Omit Semicolon School"* *(Automatic Semicolon Insertion)*
  - *"Add Semicolon School"*
    - Código mais estruturado, facilita a leitura:
    - Algumas regras:
      - usar sempre **;** que se tratar de uma expressão *top level*

```
let x=4;
```

      - não é necessário **;** no final de :
        - declaração de uma função *function name (…) {…}*
        - if (…) {…} else {…}
        - for (…) {…}
        - while (…) {…}
      - necessário **;** quando:
        - do{…} while (…);

Variáveis

JS

Variáveis

- Declaração de variáveis
  - armazenamento temporário (uma vez que após o fecho da página o browser não retém o valor atribuído à variável)
  - ***loosely typed***, não é necessário definir o tipo de variável uma vez que este é automaticamente assumido de acordo com a declaração (atribuição) efetuada
    - ***var***
    - ***let***
      - A variável só é visivel no bloco onde foi criada
    - ***const***
      - Não permite alterar a atribuição do valor inicial (declaração).

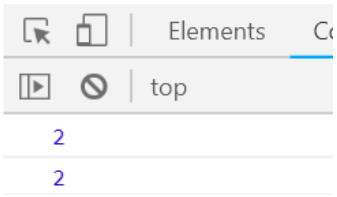
```
var x=10;       //numeric
var x="ten"     //string
var x;
```

Scope (let)

- **var** (*scope* é a função)

```
calculateSum(2);

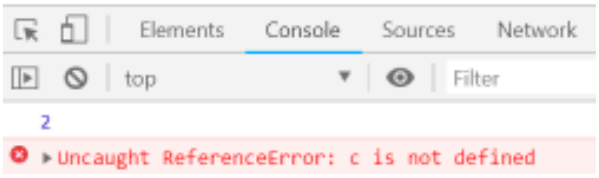
function calculateSum (a,b = 1){
    if (b==1)
        {
            var c=2;
            console.log(c);
        }
     console.log(c);
};
```



- **let** (*scope* é o bloco {…})

```
calculateSum(2);

function calculateSum (a,b = 1){
    if (b==1)
        {
            let c=2;
            console.log(c);
        }
     console.log(c);
};
```

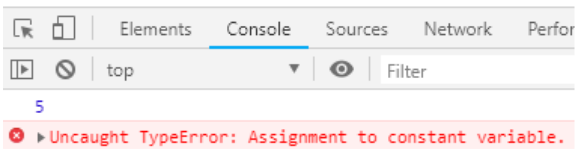


const

- **const**
  - declarar uma constante
    - sempre que se pretende atribuir a uma variável um valor que não é susceptível de ser alterado
    - frequentemente utilizado para a declaração de objetos (referência constante)

```
calculateSum(2);

function calculateSum (a,b = 1){
    const c = 5;
    console.log(c);
    c=a+b;
    console.log(c);
};
```



Variáveis

- Regras para definir o nome das variáveis:

```
var name = 'John';
var age = 26;
var isMarried = true;
```

  - Significado semântico (ex: *firstName*, …)
  - *camelCase* (convenção)
  - Podem começar por uma letra, por "$" ou por *underscore* "_".
  - Não podem conter espaços nem caracteres especiais (! . , / \ + * = …)
  - Não podem conter *keywords (*ex: var , ….)
  - Apesar de ser possível, não se devem diferenciar as variáveis apenas com base nas minúsculas e maiúsculas (ex: score e Score).

Variáveis

- O *JavaScript* permite a declaração de variáveis tendo por base dois grandes tipos:

Primitive Types	*Reference Types*
Armazenados como dados simples Contêm diretamente os valores que lhe são atribuídos	Armazenados como objetos

Primitive Types

JS

Primitive Types

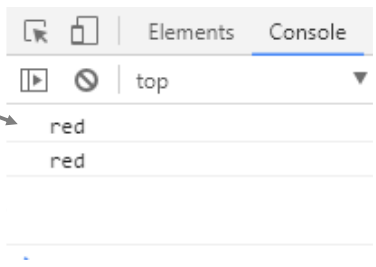
- A variável contém diretamente o valor atribuído
  - Se for igualada a outra variável o seu valor é diretamente atribuído a essa variável
    - Apesar de partilharem o mesmo valor, as variáveis são **totalmente independentes**
    - Duas localizações de memória diferentes

```
<script>
    var color1="red";
    var color2=color1;

    console.log(color1);
    console.log(color2);

    color1="blue";

    console.log(color1);
    console.log(color2);
</script>
```



Primitive Types

- O *JS* possui 5 *primitive types*:
  - number

```
var ccount=25;
```

  - string

```
var name="string exemplo";
```

  - boolean

```
var found=true;
```

  - null

```
var obj = null;
```

  - undefined
    - variável sem inicialização definida

```
var data;
```


Primitive Types

- ***number***
  - Todos os números são representados através de *floats* de 64 bits

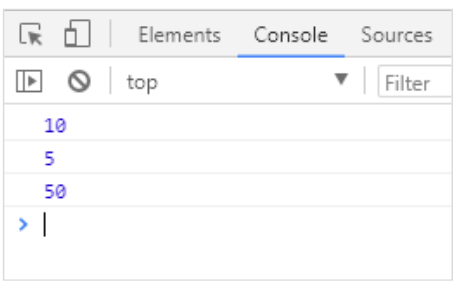
*Unlike many other programming languages, JavaScript **does not define** different types of numbers, like integers, short, long, floating-point etc.*

https://www.w3schools.com/js/js_numbers.asp

  - var num1 = 50;
  - var num2 = 10.5;
  - var num3 = 10 * 10;

```
<script>
    var price=10;
    var quantity=5;
    var total=price*quantity;

    console.log(price);
    console.log(quantity);
    console.log(total);
</script>
```

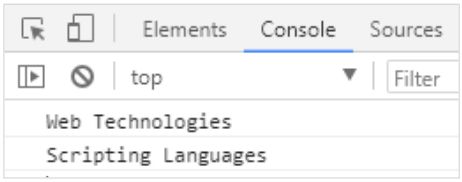


Primitive Types

- ***string***
  - Cadeia de caracteres
  - Declaração de uma string:
    - pode ser declarada com aspas **“** ou com plica **‘**, no entanto a declaração tem ser iniciada e finalizada da mesma forma
      - Quando se pretende incorporar “ ou ‘ numa string, deve declarar-se a string com o símbolo que não se pretende representar
      - Em alternativa pode recorrer-se a uma backslash \ antes da aspa ou da plica que se pretende representar.

```
<script>
    var msg1 = "Web Technologies";
    var msg2 = 'Scripting Languages';

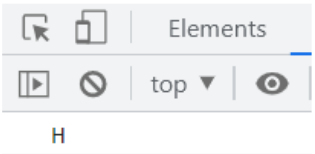
    console.log(msg1);
    console.log(msg2);
</script>
```



Primitive Types

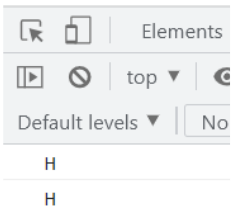
- ***string***
  - Uma *string* permite indexação, os **índices iniciam-se em 0**:

```
var s="Hello World!"
console.log(s[0])
```



  - Ao contrário de outras linguagens, ex: C, apesar de permitir indexação uma *string* não pode ser diretamente alterada

```
var s="Hello World!"
console.log(s[0])
s[0]='K'
console.log(s[0])
```

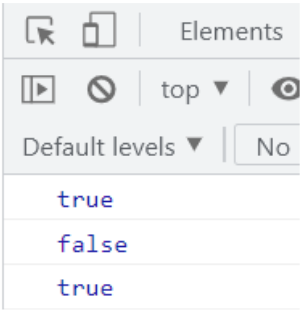


Primitive Types

- ***boolean***
  - podem assumir apenas dois valores:
    - ***true***
    - ***false***

```
var a=this;
var b=false;
var c=1;

console.log(Boolean(a));
console.log(Boolean(b));
 console.log(Boolean(c));
```



Reference Types (objects)

JS

Reference Types

- *Objeto*
  - É uma lista não ordenada de **propriedades**, consistindo num nome e num valor.
    - Quando o valor é uma função, cria-se um método.
  - Formas diferentes de criar objetos:
    - Operador ***new*** + *constructor* ***Object()***
      - *constructor* é uma função que permite a criação de um objeto com base no operador **new**
    - Forma Literal
    - Através de uma ***class***

Reference Types

- Ao contrário dos *Primitive Types* os ***Reference Types*** não guardam o objeto diretamente na variável, na realidade a variável contém um ponteiro (referência) para a localização em memória onde o objeto existe.

```
var obj1=new Object();
```



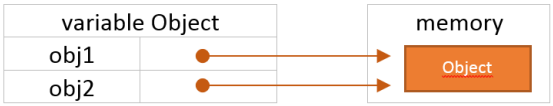
- Quando se atribui um objeto a uma variável, na realidade essa variável armazena um ponteiro que referencia o mesmo objeto
  - De facto existe apenas um objeto, o qual está a ser referenciado (apontado) por duas variáveis.

```
var obj1=new Object();
var obj2=obj1;
```



Reference Types

- O mesmo objeto referenciado por duas variáveis




```
<script>
    var obj1=new Object();

    var obj2=obj1;

    obj1.newProperty="valueOfNewProperty";

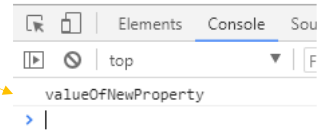
    console.log(obj2.newProperty);
</script>
```

criado **obj1**, ponteiro para 1 objeto em memória

obj2 contém ponteiro para o mesmo objeto

adicionar nova propriedade ao objeto

uma vez que também é referenciado por obj2



Reference Types

- *Declaração de Objetos*
  - *Forma Literal*
    - Propriedades são formadas por:
      - **identificador**
      - **: valor**
      - múltiplas propriedades são separadas por **virgulas**
      - termina com **};**

```
var books={
    name: "JavaScript",
    year:2014
};
```

  - A ordem das propriedades é irrelevante.

  - *new + **constructor** Object()*

*A constructor is useful when you want to **create multiple similar objects** with the same properties and methods.*

```
var books=new Object();

books.name="JavaScript";
books.year=2014;
```

Propriedades / Métodos

(objects)

(.)dot notation

- Aceder a propriedades
  - *objectName.**propertyName***

```
book={
    title:"Javascrit",
    year:2018,
    editor:"O'Reilly"
}
```

```
alert("Book title: " + book.title);
```



  - Aceder ao editor?

```
alert("Book Editor: " + book.editor);
```



(.)dot notation

- Aceder a métodos
  - *objectName.**methodName( )***

```
book={
    title:"Javascrit",
    year:2018,
    editor:"O'Reilly",

    showDetails:function(){
        return ("Book title: " + this.title + " ;     Book editor: " + this.editor);
    }
}
```

```
alert (book.showDetails());
```



Reference Types

- Alterar o valor de propriedades:

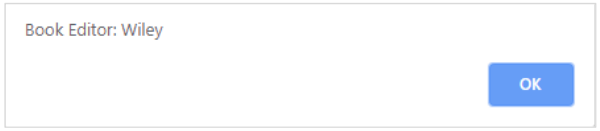
```
book={
    title:"Javascrit",
    year:2018,
    editor:"O'Reilly",

    showDetails:function(){
        return ("Book title: " + this.title + " ;     Book editor: " + this.editor);
    }
}
```

```
...
}

book.editor="Wiley";

alert("Book Editor: " + book.editor);
```



- Os objetos podem ser alterados em qualquer momento:
  - Propriedades podem ser alteradas/adicionadas/removidas

Reference Types

- Criar Propriedades/Métodos

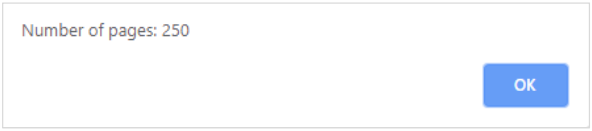
```
book={
    title:"Javascrit",
    year:2018,
    editor:"O'Reilly",

    showDetails:function(){
        return ("Book title: " + this.title + " ;     Book editor: " + this.editor);
    }
}
```

```
...
}

book.pages=250;

alert("Number of pages: " + book.pages);
```



- Os objetos podem ser alterados em qualquer momento:
  - Propriedades podem ser alteradas/adicionadas/removidas

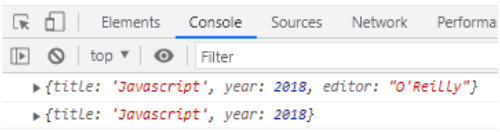
Reference Types

- Apagar propriedades/métodos

```
var book={
  title:"Javascript",
  year:2018,
  editor:"O'Reilly"
}

console.log(book)

delete(book.editor)
console.log(book)
```



- Os objetos podem ser alterados em qualquer momento:
  - Propriedades podem ser alteradas/adicionadas/removidas

Exercício 1 (properties/methods)

Exercício 1

- Criar um objeto carro, cuja propriedades são:
  - Marca: BMW
  - Cilindrada: 2000
  - Combustivel: gasoleo

- criar um método que calcule o imposto de circulação (0.05€/cc)

Exercício 1

- garanta que o valor por cc para cálculo do imposto é passado como argumento ao método calculoIuc()

- Faça variar o valor do coeficiente/cc
  - ex: 0.04/cc

Exercício 1

- altere o valor da propriedade marca de 'BMW' para 'Mercedes', sem alterar a declaração original do objeto.

- Adicione a propriedade pintura com o valor "metalizada"

Built-in Types (Reference Types)

Reference Types

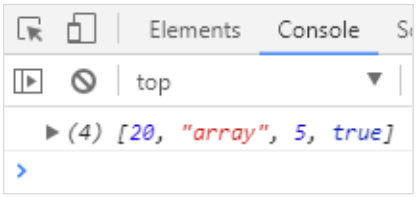
- *Built-in types*
  - Criar *objets* com o constructor (keyword **new**)
    - Object `const books = new Object( )`
    - Array `const items = new Array( )`
    - Date
    - Error
    - Function
    - …
  - Os *built-in types* podem ter formas literais
    - Sintaxe literal permite a criação de objetos sem utilizar o operador ***new*** e o respetivo *constructor*

```
const items = [ ];
```

Reference Types

- *Array*
  - Permite armazenar um conjunto de valores relacionados
  - Ao contrário de outras linguagens:
    - O *array* não tem de ser declarado com uma dimensão
    - Inclui **diferentes tipos de dados** no mesmo *array*
  - Notação literal
    - Definidos com **[…]** e elementos separados por virgulas

```
const values=[20,'array',5, true];
console.log(values);
```



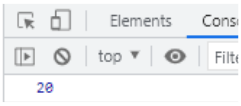
    - Baseado num *constructor*:

```
const valuesConstructor=new Array(20,'array',5,true)
console.log(valuesConstructor)
```


Array

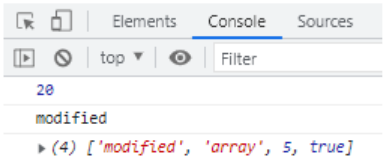
- Indexação
  - nomeArray [***posição***]
  - Índices iniciam-se em ***zero***

```
const values=[20,'array',5, true];
console.log(values[0]);
```



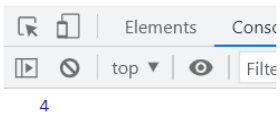
  - Um array pode ser diretamente alterado

```
values[0]='modified';
console.log(values[0]);
console.log(values);
```



  - Propriedade ***length*** é muito importante (retorna a dimensão do *array*)

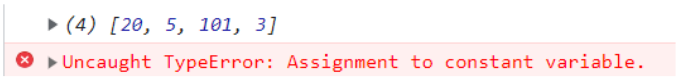
```
console.log(values.length);
```



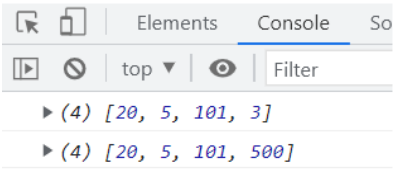
Array

- É usual declarar um array com base em const
  - Quando se define o array como const, na realidade define-se uma referência constante para o array o que não impede de alterar os seus elementos

```
const a=[20,5,101,3]
 console.log(a)
 a=[5,4]
console.log(a)
```




```
const a=[20,5,101,3]
console.log(a)
a[3]=500
console.log(a)
```



  - previne que a referência para o objeto não é alterada **o que não impede** a alteração das suas propriedades

Array

JavaScript Array Methods and Properties

Name	Description
concat()	Joins arrays and returns an array with the joined arrays
constructor	Returns the function that created the Array object's prototype
copyWithin()	Copies array elements within the array, to and from specified positions
entries()	Returns a key/value pair Array Iteration Object
every()	Checks if every element in an array pass a test
fill()	Fill the elements in an array with a static value
filter()	Creates a new array with every element in an array that pass a test
find()	Returns the value of the first element in an array that pass a test

... ...

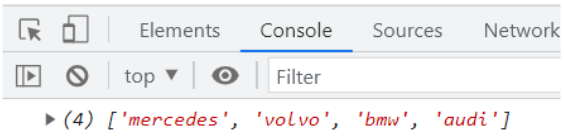
https://www.w3schools.com/jsref/jsref_obj_array.asp

Exercício 2 (Array object)

Exercício 2

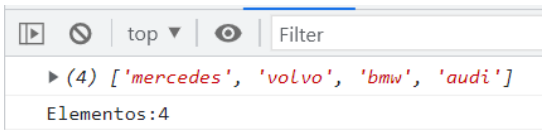
- Declare um array com os valores: mercedes, volvo, bmw, audi

```
const cars=['mercedes','volvo','bmw','audi']
console.log(cars)
```



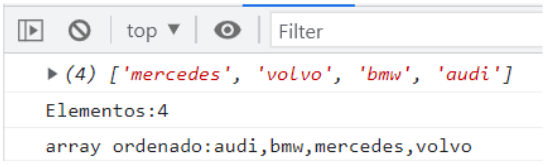
- Determine qual o número de elementos do array (propriedade length)

```
console.log('Elementos:'+cars.length)
```



- Ordene o array (método: sort())

```
console.log('array ordenado:'+cars.sort())
```



- *Functions*
  - A forma literal é muito mais usada para a declaração de funções do que baseada num constructor, uma vez que é menos sujeita a erros e mais fácil de manter

```
function reflect(value){

  return value;
}
```



  - Tendo por base um constructor seria:
    - Todo o corpo da função teria de ser encapsulado numa string, o que como é óbvio tem várias desvantagens (dificulta a implementação, dificulta o debug do código, …)

```
var reflect=new Function("value","return value;");
```



  - À excepção do *built-in type Function* (notação literal) não existe uma forma correta ou errada de instanciar *built-in types*.

Primitive Wrapper Types

Primitive Wrapper Types

- Existem três ***Primitive Wrapper Types***:
  - *String*
  - *Number*
  - *Boolean*
    - Possibilitam o funcionamento com os *Primitive Types* da mesma forma ***(dot notation)*** que ocorre com os *Reference Types*
    - Ao contrário dos *reference type*, um *primitive wrapper type* **não permite a adição de propriedades.**

Strings

String Properties

Property	Description
constructor	Returns the string's constructor function
length	Returns the length of a string
prototype	Allows you to add properties and methods to an object

String Methods

Method	Description
charAt()	Returns the character at the specified index (position)
charCodeAt()	Returns the Unicode of the character at the specified index
concat()	Joins two or more strings, and returns a new joined strings
endsWith()	Checks whether a string ends with specified string/characters
fromCharCode()	Converts Unicode values to characters
includes()	Checks whether a string contains the specified string/characters
indexOf()	Returns the position of the first found occurrence of a specified value in a string
lastIndexOf()	Returns the position of the last found occurrence of a specified value in a string

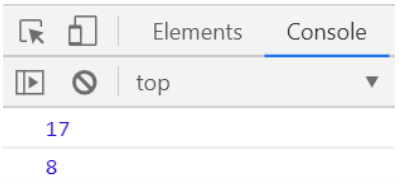
https://www.w3schools.com/jsref/jsref_obj_string.asp

Array

- Propriedade (exemplo: length)

```
<script>

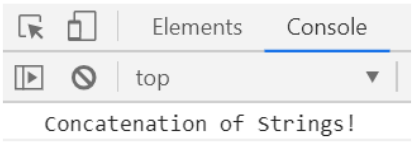
    let firstStr='Concatenation of ';
    let secondStr='Strings!';
    console.log(firstStr.length);
    console.log(secondStr.length);
```



- Método (exemplo: concat())

```
<script>

    let firstStr='Concatenation of ';
    let secondStr='Strings!';
    console.log(firstStr.concat(secondStr));
```



Variáveis

Primitive Types	Reference Types (Built-in)	Primitive Wrapper Types
Number	Object	Number
String	Array	String
boolean	Date	Boolean
null	Error	
undefined	Function	
	RegExp	
	Math	
	...	

Operadores

JS

Operadores

- Aritméticos

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

http://www.w3schools.com/js

Operadores

- Atribuição

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

http://www.w3schools.com/js

Operadores

- Lógicos

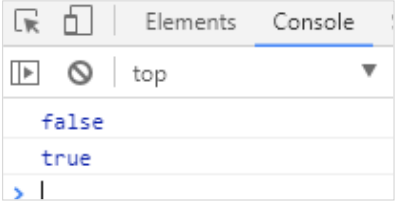
Operator	Description
&&	logical and
||	logical or
!	logical not

http://www.w3schools.com/js

```
<script>
    var a=5;
    var b=4;
    var c=6;
    var d=8;

    console.log((a>b)&&(c>d));

    console.log((a>b)||(c>d));
</script>
```



Operadores

- Comparação

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

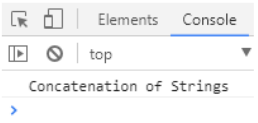
https://www.w3schools.com/js/js_comparisons.asp

Operadores

- Strings
  - Concatenação (+)
    - Operador muito frequentemente utilizado

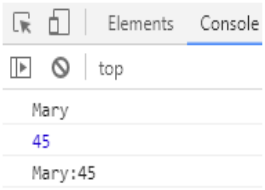
```
<script>
  var strC = "Concatenation " + "of" + " Strings";
  console.log(strC);
</script>
```

Concatenação



    - *type coercion* (um dos argumentos é uma *string*):

```
console.log(name);
console.log(age);
console.log(name + ":" + age);
```



Operadores

- Precedência de Operadores

The following table is ordered from highest (20) to lowest (1) precedence.

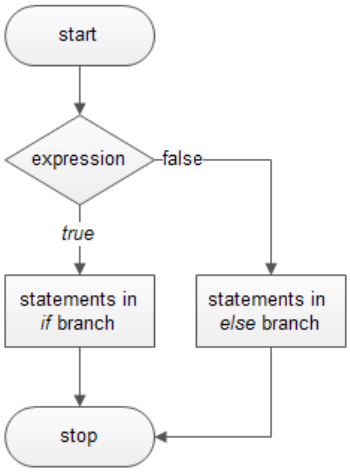
Precedence	Operator type	Associativity	Individual operators
20	`Grouping`	n/a	`( … )`
19	`Member Access`	left-to-right	`… . …`
	`Computed Member Access`	left-to-right	`… [ … ]`
	`new` (with argument list)	n/a	`new … ( … )`
	Function Call	left-to-right	`… ( … )`
18	`new` (without argument list)	right-to-left	`new …`
17	`Postfix Increment`	n/a	`… ++`
	`Postfix Decrement`		`… --`
16	Logical NOT	right-to-left	`! …`
	Bitwise NOT		`~ …`
	Unary Plus		`+ …`
	Unary Negation		`- …`
	Prefix Increment		`++ …`
	Prefix Decrement		`-- …`
	typeof		`typeof …`

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_Precedence

Estruturas de Seleção (condicionais)

JS

Seleção




```
<script type="text/javascript">
     var teste = "verdadeiro";

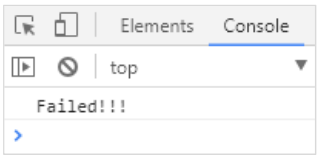
     if (teste == "verdadeiro")
         document.write("Condição Verdadeira!");
     else
         document.write("Condição Falsa!");
 </script>
```


Seleção

- if …else

```
<script>
    var threshold=50;
    var grade=40;

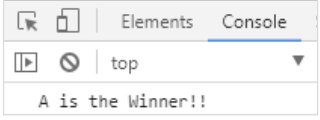
    if (grade>=threshold)
        {console.log("Aproved!!");}
    else
        {console.log("Failed!!");}
</script>
```



- condições encadeadas

```
<script>
    var scoreA=60;
    var scoreB=50;

    if (scoreA>scoreB)
        {console.log("A is the Winner!!");}
    else if (scoreA<scoreB)
        {console.log("B is the Winner!!");}
    else
        {console.log("Draw !!")}
</script>
```



Seleção

- switch (var)

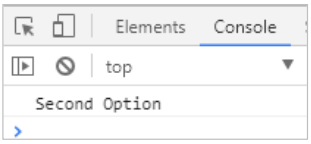
  { case value1: statements; break;

  case value2: statements; break; …

  default: statements }

```
<script>
    var msg,a;
    a=2;

    switch(a)
    {
        case (1): msg="First Option"; break;
        case (2): msg="Second Option"; break;
        case (3): msg="Third Option"; break;
        default: msg="No option!"; break;
    }
    console.log(msg);
</script>
```



Estruturas de Repetição

JS

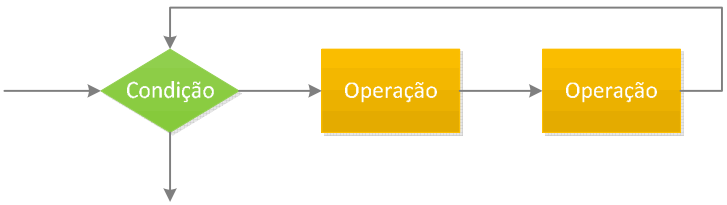
Repetição

- Estruturas de Repetição / Ciclos *(Loops)*

- ***while (condição){***

  // código bloco

  }



- ***do {***

  // código bloco

  } while (condição);



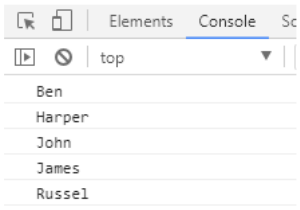
Repetição

- Ciclos *(Loops)*

  - ***for*** *(initialization; condition; variable update) { …}*

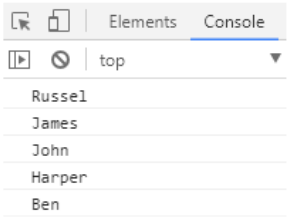
```
names=["Ben", "Harper","John", "James", "Russel"];

for(i=0; i<names.length; i++)
    {
        console.log(names[i]);
    }
```




```
names=["Ben", "Harper","John", "James", "Russel"];

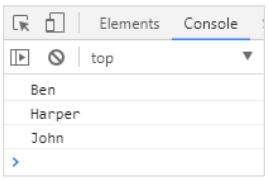
for(i=names.length-1;i>=0;i--)
    {
        console.log(names[i]);
    }
```



Repetição

- Ciclos *(Loops)*
  - ***break***

```
for(i=0; i<names.length; i++)
    {
        console.log(names[i]);
        if(i===2)
            {break;}
    }
```

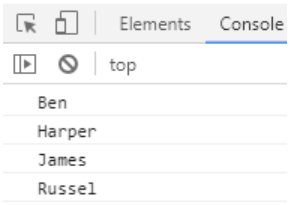


break: interrompe o funcionamento do ciclo

  - ***continue***

```
names=["Ben", "Harper","John", "James", "Russel"];

for(i=0; i<names.length; i++)
    {
        if(i===2)
            {continue;}
        console.log(names[i]);
    }
```



continue: salta diretamente para o final da iteração e prossegue com o ciclo, neste caso ignora a 3ª iteração

falsy / truthy

- *falsy*
  - valores tratados como ***false***

Value	Description
var highScore = false;	false
var highScore = 0;	0
var highScore = ‘ ’;	*empty value*
var highScore = 10/’score’;	*NaN (not a number)*
var highScore;	variável sem valor atribuído

- *truthy*
  - valores tratados como ***true***

Value	Description
var highScore = true;	true
var highScore = 1;	número ≠ 0
var highScore = ‘xxxx ’;	string com conteúdo
var highScore = 10/5	*resultado de um cálculo* ≠ 0

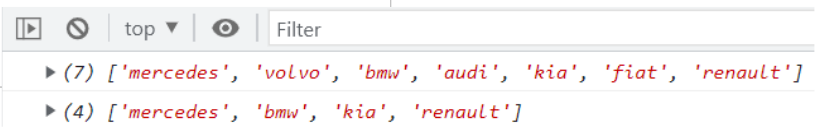
J. Ducket, JavaScript and jQuery, 2014

Exercício 3 (Repetição)

Exercício 3

- Crie um array com a designação cars, contendo os valores:
  - 'mercedes','volvo','bmw','audi','kia','fiat','renault'
- crie um novo array (newcars) com base no array anterior, o qual contém apenas os valores correspondentes às posições pares
  - deve utilizar o método push()

```
const cars=['mercedes','volvo','bmw','audi','kia','fiat','renault']
const newCars=[]
for (let i=0; i<cars.length;i++ )
    {
     if(i%2==0)
       newCars.push(cars[i])
    }
console.log(cars)
console.log(newCars)
```



Funções

JS

Funções

- Conjunto de declarações agrupadas para executar uma tarefa específica.
  - Reutilização de código; flexibilidade; …
    - **Declaração de uma função** (notação literal):

function name (param1, param2, ….){

código a ser executado;}

```
function  firstFunction(){
    document.write("hello");
}
```

    - Prefixos uteis para nomes de função:
      - create, show, get, check, ….

Funções

- Chamada à função:
  - Efetuada através do nome da função seguido de parêntesis
  - Código só é executado após a respetiva chamada

```
firstFunction();
```

- O browser percorre todo o script antes da execução de cada declaração, mas preferencialmente a função deve ser declarada antes da sua chamada.

Funções

- Parâmetros
  - Declaração de uma função com parâmetros:

```
function  calculateArea(width,height){
        return width*height;
}
```

  - Chamada a uma função:
    - Especificação direta dos valores dos argumentos

```
calculateArea (2,4);
```

    - Argumentos da função definidos através de variáveis

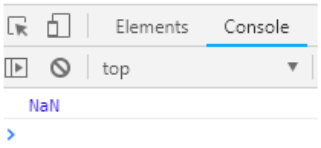
```
rectWidth=2;
rectHeight=4;
calculateArea(rectWidth, rectHeight);
```

Funções

- Possível definir *default values* para os parâmetros

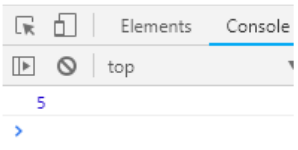
```
function calculateSum (a,b){
            return a+b;
        };

console.log(calculateSum(4));
```




```
function calculateSum (a,b = 1){
          return a+b;
      };

console.log(calculateSum(4));
```



Funções

- Retorno de um valor único:

```
function  calculateArea(width,height){
    return width*height;
}
var wallOne=calculateArea(3,5);
var wallTwo=calculateArea(8,5);
```

- Retorno de vários valores com base em *arrays*:

```
<script>

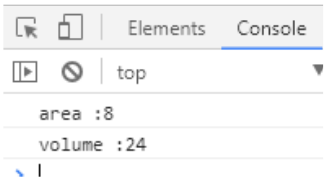
   function  getDimensions(w,h,d)
       {   var area=w*h;
           var volume=area*d;
           var values=[area,volume];

           return values;}

   console.log("area : " + getDimensions(2,4,3)[0]);

   console.log("volume : " + getDimensions(2,4,3)[1]);

</script>
```



Passagem de Parâmetros

- Os *Primitive Type* e os Objetos são passados à função de forma diferente
  - Nos *Primitive Types* é feita a ***passagem do valor*** do argumento:
    - todas as alterações efetuadas no parâmetro no interior da função não alteram o valor original
  - Nos *Reference Types* a passagem é ***feita por referência:***
    - as alterações feitas no interior da função são na realidade efetuadas no objeto original (objeto é referenciado pelas diversas variáveis)

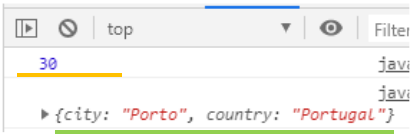
```
<script>
    var age=30;
    var citizen={city:'Coimbra', country:'Portugal'};

    function changeValues(a,b){
        a=50;
        b.city='Porto'
    }

    changeValues(age,citizen);

    console.log(age);
    console.log(citizen);

</script>
```

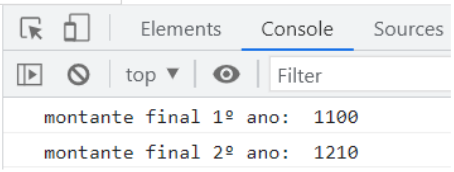


Exercício 4 (funções)

Exercicio 4

- Declare uma função que calcule a valorização de um depósito ao fim do primeiro e do segundo ano.
  - A função recebe o montante investido e a taxa de juro (fixa) e deve mostrar o montante ao fim do 1º e do 2º ano. (ex: montante 1000€, juros 10%)

```
function calculaJuros(m,j){
    const values=[];
    j/=100;
    values[0]=m*(1+j);
    values[1]=values[0]*(1+j);
    return values
  }
console.log('montante final 1º ano:  '+ calculaJuros(1000,10)[0])
console.log('montante final 2º ano:  '+ calculaJuros(1000,10)[1])
```



Diferentes Tipos de Função

JS

Funcões

- O JavaScript admite formas diferentes de criar uma função:

Declaração de Função

(statement)

```
function calculateSum (a,b = 1){
          return a+b;
      }
```

Function Expression

(anonymous function)

```
var calculateSum = function (a,b = 1){
          return a+b;
      }
```

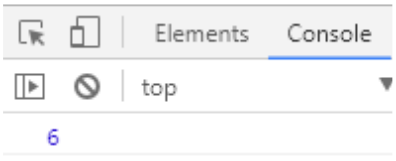

Funções

- Declaração de Função
  - Chamada à função:

```
<script>
    var area;
    area=calculaArea(2,3);

    function calculaArea(width,height)
            {
            return width*height;}


    console.log(area);
</script>
```



A declaração normal de uma função permite que a chamada à função seja **executada antes** da declaração da função

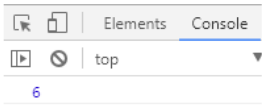
Funções

- *Function Expression (anonymous function)*
  - A declaração de uma função pode ser incorporada numa expressão
    - Não é especificado o nome da função depois de ***function (anonymous function)***

```
<script>
    var calculaArea=function(width,height){
                        return width*height;}

    var area=calculaArea(2,3);
    console.log(area);
</script>
```

A declaração de uma ***anonymous function*** exige que a chamada à função seja efetuada **depois da** expressão



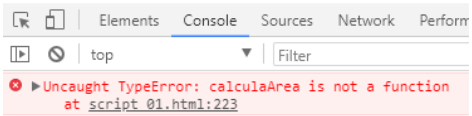
    - É tratada como uma expressão, ou seja a função é detetada só após a sua chamada

```
<script>
    var area=calculaArea(2,3);

    var calculaArea=function(width,height){
        return width*height;}

    console.log(area);
</script>
```

Área não calculada, uma vez que a chamada à função foi feita antes da expressão onde está declarada



Funções

- A ***anonymous function (function expression)*** é particularmente importante no Javascript
  - definição de métodos de um objeto
  - event handling

```
var course={
    name:"web technologies",
    displayName:function(){
        document.write(course.name);
    }
}
course.displayName();
```

anonymous function *define o método displayName*

A definição de uma propriedade (nome:valor) é igual à definição de um método (nome:valor), neste último o valor é uma ***anonymous function***



Funcões

- O JavaScript admite formas diferentes de criar uma função:

Declaração de Função

```
function calculateSum (a,b = 1){
          return a+b;
      }
```

Function Expression

(anonymous function)

```
var calculateSum = function (a,b = 1){
          return a+b;
      }
```

- Sintaxe abreviada
- Palavra **function** é eliminada
- A seta **=>** aponta para o corpo da função
- Caso não existam parâmetros são necessários parênteses vazios
- Se a função possuir várias declarações são necessárias chavetas e a key word **return**

Arrow Functions

```
var calculateSum = (a, b = 1) =>  a+b;
```


Redução do Acoplamento

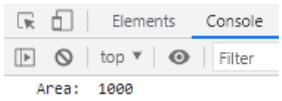
Keyword *this*

this

- A keyword ***this*** pertence a um determinado *scope*, isto é:
  - Se usado num **método de um objeto** a *keyword* ***this*** refere-se a esse objeto
    - Se o objeto é criado com base numa class a utilização do *this* refere-se a cada instância em particular

```
const shape={
    width:50,
    height:20,
    calculateArea:function(){
      return this.width*this.height
    }
  }
  console.log('Area:  ' + shape.calculateArea())
```

width / height do objeto shape



this

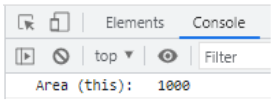
- Se aplicado numa função destinada a definir o método de um objeto, *this* refere-se ao objeto que contém o método

```
var width=100;
const shape={
   width:50,
   height:20
}

var calculateArea=function(){
  return this.width*this.height
}

shape.getArea=calculateArea
console.log('Area (this):   '+ shape.getArea())
```

this refere-se ao objeto **shape**, uma vez que a função **calculateArea** é usada para definir o método **getArea** do objeto **shape**



- Se aplicado isolado ou numa função genérica, **this** refere-se ao global object (window object)
- Se aplicado num evento, **this** refere-se ao elemento que recebe o evento

Scope das Variáveis

Scope

- Variável Global
  - Criada fora de qualquer função
  - É visível em todo o script (contexto global)
  - Armazenamento permanente (a variável existe enquanto a página estiver ativa no browser)
    - requer mais memória que as variáveis locais (eliminadas após a execução a função onde foram declaradas)
    - possíveis conflitos na nomenclatura das variáveis
- Variável Local
  - Criada no interior de uma função
  - Só é visível na função onde foi criada
  - Uma vez terminada a execução da função a variável é eliminada

Scope

- Cada nova função cria um *scope*
  - Um espaço/ambiente onde as respetivas variáveis são acessíveis

```
var a="Web ";
first();

function first(){
    var b="Technologies: ";
    second();

    function second(){
        var c="HTML; CSS; JS";
        console.log(a + b + c);
    }
}
```

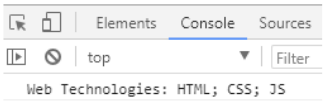
Global scope
a="Web "

first() scope
a="Web "
b="Technologies: "

second() scope
a="Web "
b="Technologies: "
c="HTML; CSS; JS"

Scope Chain

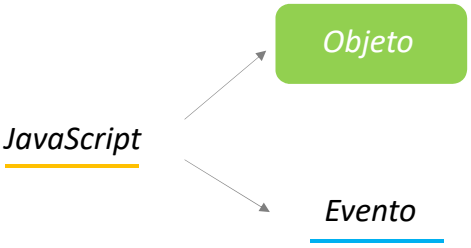
- Uma função que é **definida no interior** de uma outra função, tem **acesso ao scope da função que a envolve** *(scope chain)*



https://www.udemy.com/the-complete-javascript-course/learn/v4/t/lecture/5869134?start=600

Criação de Objetos JS

Objetos



JavaScript Objects

In JavaScript, objects are king. If you understand objects, you understand JavaScript.

In JavaScript, almost "everything" is an object.

- Booleans can be objects (or primitive data treated as objects)
- Numbers can be objects (or primitive data treated as objects)
- Strings can be objects (or primitive data treated as objects)
- Dates are always objects
- Maths are always objects
- Regular expressions are always objects
- Arrays are always objects
- Functions are always objects
- Objects are objects

In JavaScript, all values, except primitive values, are objects.

Primitive values are: strings ("John Doe"), numbers (3.14), true, false, null, and undefined.

http://www.w3schools.com/js/js_object_definition.asp

Objetos

- Um objeto representa uma entidade física ou conceptual.
  - Tem estado **(propriedades)**, comportamento **(métodos) e** identidade **(nome)**.

- Propriedades
  - Valores associados ao objeto

- Métodos
  - Ações associadas ao objeto
  - Mesma sintaxe que propriedades
    - O valor é uma função

```
<script>
    var hotel = {

        name: 'Coimbra',
        rooms: 20,
        booked: 15,
        gym: true,
        roomTypes:['single', 'double','suite'],

        checkAvailability: function () {
            return this.rooms - this.booked;
        }

    }
</script>
```

Objetos

3 formas distintas de criar objetos

- Forma Literal
- new + Object()
- class

Objetos

Forma Literal

```
const hotel={

    name:'Coimbra',
    rooms:20,
    booked:15,
    gym:true,
    roomTypes:['single','double','suite'],

    checkAvailability:function(){
      return this.rooms - this.booked
    }
  }
```

Nome do objeto

Propriedades *em que os valores podem ser:*
- *string*
- *number*
- *boolean*
- *array*

Métodos
O scope do **this** é o objeto hotel

Objetos

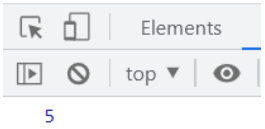
new + Object()

```
const hotel= new Object();

hotel.name='Coimbra';
hotel.rooms=20;
hotel.booked=15;
hotel.gym=true;
hotel.roomTypes=['single','double','suite'];

hotel.checkAvailability=function(){
        return this.rooms - this.booked}

console.log(hotel.checkAvailability());
```



5

- cria um objeto ao qual **se adicionam propriedades e métodos**
- sintaxe totalmente diferente da forma literal.

Objetos

class

- Definição de uma *class*: "A class is a blueprint for objects"
  - Todas as classes possuem um ***constructor*** (método) onde se inicializam as propriedades

```
class Player{
    constructor(firstName,lastName){
      this.fname=firstName,
      this.lname=lastName
    }
    showName(){
      console.log('player name: ' + this.fname + this.lname)
    }
  }
```

nome da Classe (Por convenção o nome da class inicia-se com maiúscula)

constructor (propriedades)

método

Objetos

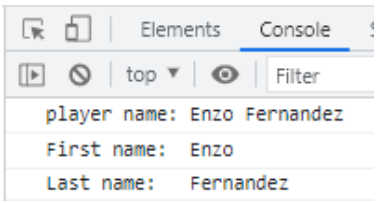
class

- Criar um objeto passa pela criação de uma **instância da class**

```
const playerInstance=new Player('Enzo',' Fernandez')

playerInstance.showName()

console.log('First name:  ' + playerInstance.fname)
console.log('Last name:  ' + playerInstance.lname)
```



Objetos

- As classes permitem herança
  - uma classe pode fazer o ***extend*** de outra classe (herdar as suas propriedades e métodos)

```
class Person{
    constructor(a,b){
      this.age=a,
      this.country=b }
    showFeatures(){
      console.log('Age:  '+ this.age + '  Country:  ' + this.country)
    }
  }

  class Player extends Person{
    constructor(firstName,lastName,c, d){
      super(c,d)
      this.fname=firstName,
      this.lname=lastName}
    showName(){
      console.log('player name: ' + this.fname + this.lname) }
  }
```

Person

Player

super() na classe filha é obrigatório, executado para invocar o constructor da classe pai

- Herança

```
const playerInstance=new Player('Enzo',' Fernandez', 21, 'Argentina')

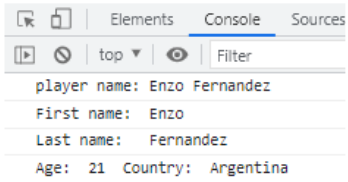
playerInstance.showName()

console.log('First name:  ' + playerInstance.fname)
console.log('Last name:  ' + playerInstance.lname)

playerInstance.showFeatures()
```

Player

Person



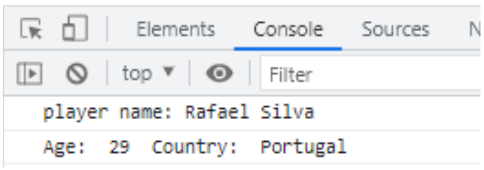
Exercício 5(Objeto)

Objetos

- Com base no exemplo anterior, crie uma nova instância da class Player.
  - Deve mostrar na consola o nome completo do jogador assim como a idade e o respetivo país.
    - Considere os métodos showName() e show Features()

```
const playerInstance=new Player('Rafael','Silva', 29, 'Portugal')

playerInstance.showName()
playerInstance.showFeatures()
```



Criação de Objetos em JS

Resumo

Objetos

Forma Literal

- Pouco eficiente quando se pretendem criar múltiplas instâncias, obriga a definição repetida dos métodos
- Sintaxe especifica
- Permite a criação de um objeto vazio ao qual se adicionam propriedades e métodos

new + Object()

- Pouco eficiente quando se pretendem criar múltiplas instâncias

class

- Definir uma classe e criar múltiplas instâncias dessa classe
- Solução mais eficaz quando se pretende a definição de múltiplos objetos (instâncias)
- Só disponível no ES6

Propriedades / Métodos

Resumo

(.)dot notation

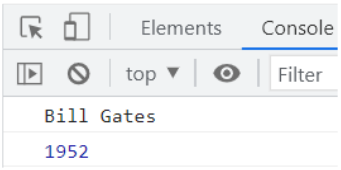
- Propriedades
  - *objectName.**propertyName***

```
const bill={
    name: 'Bill Gates',
    age:70,
    height:185,
    calculateYearBirth: function(a){
     return a-this.age;
    }
  }
```

```
console.log(bill.name);
```

- Métodos
  - *objectName.**methodName( )***

```
console.log(bill.calculateYearBirth(2022));
```



Reference Types

- Adicionar Propriedades
  - As propriedades podem ser adicionadas quando o objeto é criado ou então posteriormente em qualquer momento
    - Por defeito, em JavaScript os objetos podem ser sempre modificados

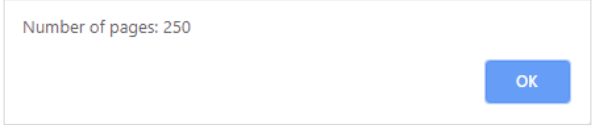
```
book={
    title:"Javascrit",
    year:2018,
    editor:"O'Reilly",

    showDetails:function(){
        return ("Book title: " + this.title + " ;     Book editor: " + this.editor);
    }
}
```

```
...
}

book.pages=250;

alert("Number of pages: " + book.pages);
```



Objetos

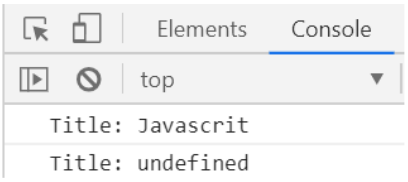
- Remover propriedades
  - Da mesma forma que é possível criar também é possível remover propriedades em qualquer altura através do operador ***delete***

```
book={
        title:"Javascrit",
        year:2018,
        editor:"O'Reilly"
    }

console.log('Title: ' + book.title);

delete book.title;

console.log('Title: ' + book.title);
```



  - A propriedade foi removida como tal não se encontra definida (*undefined)*
  - Existem métodos que impedem a possibilidade de alterações nas propriedades de um objeto:
    - *Object.preventExtensions() ; Object.seal(); Object.freeze()*

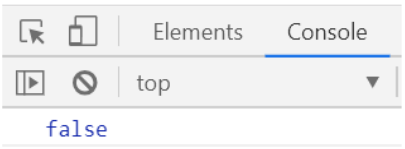
Objetos

- Detetar propriedades
  - Existem várias formas de detetar propriedades mas a forma mais fiável passa por utilizar o operador ***in***

```
book={
        title:"Javascrit",
        year:2018,
        editor:"O'Reilly"
    }


delete book.title;

console.log('title' in book);
```



JavaScript Built-in Objects

JavaScript Built-in Objects

JavaScript Reference

The references describe the properties and methods of all JavaScript objects, along with examples.

Array	Boolean	Date	Error	Global
JSON	Math	Number	Operators	RegExp
Statements	String			

https://www.w3schools.com/jsref/default.asp

- String (*Primitive Wrapper Types)
- Number (*Primitive Wrapper Types)
- Math
- Date
- Array
- Boolean (*Primitive Wrapper Types)
- RegExp
- …

Standard Built-in Objects

JavaScript Built-in Objects: String

- *String*

Propriedade	*Descrição*
length	retorna o número de caracteres que constituem a string

Método	*Descrição*
toUpperCase()	Conversão para maiúsculas
toLowerCase()	Conversão para minúsculas
trim()	Remove espaços em branco do inicio e do fim da string
split()	Permite dividir uma string e guardar cada componente numa string
replace()	Considera um valor que deve ser substituído por outro
substring()	Retorna os caracteres entre dois índices
charAt()	Retorna um caracter numa determinada posição
indexOf()	Retorna a posição da primeira ocorrência de um dado valor na string, retorna -1 se o valor nunca é detetado. É case sensitive.

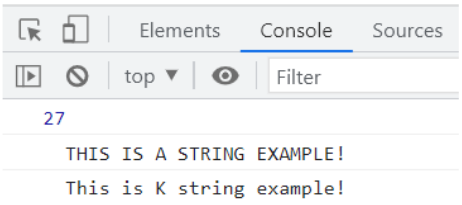
http://www.w3schools.com/jsref/jsref_obj_string.asp

Exercício 5 (propriedades/Métodos String)

- declare a string “This is a string example!”
- Mostre o número de caracteres
- Converta a string para maiúsculas
- Substitua o ‘a’ por ‘K’

```
var s="  This is a string example!"

console.log(s.length)
console.log(s.toUpperCase())
console.log(s.replace(('a'),'K'))
```



JavaScript Built-in Objects: Number

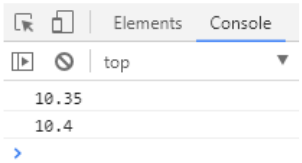
- Number

Método	Descrição
toExponential()	conversão para notação exponencial
toPrecision()	arredonda o número para um dado nº de dígitos
toFixed()	arredonda o número para um dado nº de casas decimais
toString()	converte para uma *string*
…	…

```
<script>

    var x=10.354;

    console.log(x.toFixed(2));
    console.log(x.toPrecision(3));
</script>
```



JavaScript Built-in Objects: Math

- Math
  - Operações matemáticas, disponibiliza funções matemáticas avançadas (trigonometria, estatística, etc.)
  - As propriedades/métodos do Math são chamadas **sem criar de forma explicita um objecto** do tipo Math (exemplo: Math.round(x)) **Exceção!**

Propriedade	Descrição
Math.PI	retorna aproximadamente 3.14159265359

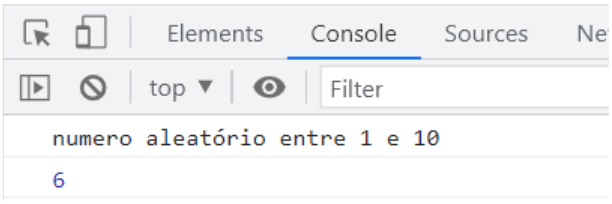
Método	Descrição
Math.round()	arredonda o número para o inteiro mais próximo
Math.sqrt()	Raiz quadrada de um número positivo
Math.ceil()	arredonda o número para o inteiro imediatamente seguinte
Math.floor()	arredonda o número para o inteiro imediatamente anterior
Math.random()	Gera um número aleatório entre 0 e 1

Exercício 6 (propriedades/Métodos Math)

Exercício 6

- Gera um número aleatório entre 1 e 10
  - *Math.random()* (gera um número aleatório no interval [0,1])
  - * (multiplicação)
  - + (adição)

```
var num = Math.floor(Math.random()*10)+1
console.log('numero aleatório entre 1 e 10')
console.log(num)
```



JavaScript Built-in Objects: Date

JavaScript Built-in Objects

- Date
  - Disponibiliza métodos e atributos para aceder e manipular horas e datas
  - **É necessário instanciar** um objeto deste tipo para aceder aos seus métodos

```
var data = new Date();  //sem parâmetros o objeto é criado com  a data atual
```

JavaScript Date Methods and Properties

Name	Description
constructor	Returns the function that created the Date object's prototype
getDate()	Returns the day of the month (from 1-31)
getDay()	Returns the day of the week (from 0-6)
getFullYear()	Returns the year
getHours()	Returns the hour (from 0-23)
getMilliseconds()	Returns the milliseconds (from 0-999)
getMinutes()	Returns the minutes (from 0-59)
getMonth()	Returns the month (from 0-11)
getSeconds()	Returns the seconds (from 0-59)

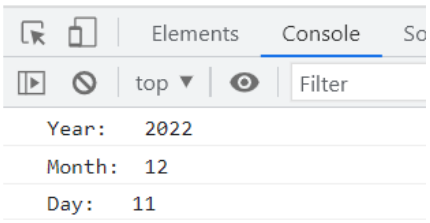
http://www.w3schools.com/js/js_obj_date.asp

Exercício 7 (propriedades/Métodos Date)

Exercício 7

- Instancie um novo objeto do tipo Date (obtendo a data atua)
- Mostre na consola:
  - ano: getFullYear())
  - mês: getMonth()
  - dia: getDate()

```
const  tdDate = new Date()
console.log('Year:   ' + tdDate.getFullYear())
console.log('Month:  ' + (tdDate.getMonth()+1))
console.log('Day:    ' + tdDate.getDate())
```



JavaScript Built-in Objects: Array

JavaScript Built-in Objects

- *Array*
  - permite armazenar diferentes tipos de elementos

Propriedade	Descrição
length	número de elementos de um *array*

Método	Descrição
indexOf()	pesquisa um elemento e retorna a sua posição
pop()	remove o último elemento de um *array*
push()	adiciona um elemento ao *array*
shift()	remove o primeiro elemento de um *array*
sort()	ordena os elementos de um *array*
unshift()	adiciona elementos no inicio de um *array*
…	…

https://www.w3schools.com/jsref/jsref_obj_array.asp

JavaScript Built-in Objects

- *Arrays*
  - Os *arrays* são particularmente importantes em *JS* precisamente pela capacidade de armazenar valores relacionados.
    - declarados de forma literal ou com base no *constructor Array*
    - índice iniciam em zero

```
<script>

    var names = ['John', 'Jane', 'Mark'];
    var years = new Array(1990, 1969, 1948);

    console.log(names[2]);

    names[1] = 'Ben';
    console.log(names);

</script>
```



Exercício 8 (propriedades/Métodos Array)

JavaScript Built-in Objects

- Declare um array com os valores

  "Audi", "Mercedes", "Volvo", "BMW"

```
const cars=['Audi', 'Mercedes', 'Volvo', 'BMW']
```

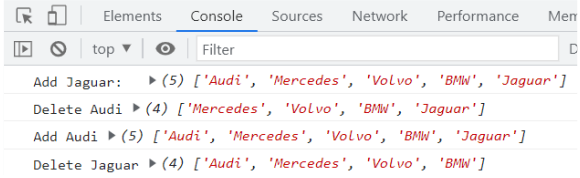
- Adicione 'Jaguar' no final: push()
- Apague o primeiro elemento: shift()
- Adicione o primeiro elemento: unshift()
- Apague o ultimo elemento: pop()

```
cars.push('Jaguar')
console.log('Add Jaguar:  ', cars)

cars.shift()
 console.log('Delete Audi', cars)

cars.unshift('Audi')
 console.log('Add Audi',cars)

cars.pop()
console.log('Delete Jaguar',cars)
```



JavaScript Built-in Objects

- *map()*
  - Executa uma função para cada um dos elementos do array

```
const numbers =[1,2,3,4];

const newNumbers = numbers.map((num)=>{return num*2;})

console.log(numbers);
console.log(newNumbers);
```

```
▶ (4) [1, 2, 3, 4]
▶ (4) [2, 4, 6, 8]
>
```

- *find()*
  - Retorna o valor do primeiro elemento que verifica a condição

```
const numbers =[1,2,3,4];

const newNumbers = numbers.find((num)=>{return num>2;})

console.log(numbers);
console.log(newNumbers);
```

```
▶ (4) [1, 2, 3, 4]
3
>
```


JavaScript Built-in Objects

- *findIndex()*
  - Retorna o indíce do primeiro elemento que verifica a condição. Caso não exista nenhum elemento que verifique a condição retorna -1.

```
const numbers =[1,2,3,4];

const newNumbers = numbers.findIndex((num)=>{return num === 2;})

console.log(numbers);
console.log(newNumbers);
```

```
▶ (4) [1, 2, 3, 4]
1
>
```

- *filter()*
  - Cria um novo array com todos os elementos que verificam a condição.

```
const numbers =[1,2,3,4];

const newNumbers = numbers.filter((num)=>{return num > 2;})

console.log(numbers);
console.log(newNumbers);
```

```
▶ (4) [1, 2, 3, 4]
▶ (2) [3, 4]
>
```

- *reduce()*
  - Executa uma função definida pelo utilizador em cada elemento do array e cujo resultado é um valor único (ex: a soma de todos os elementos).

```
const numbers =[1,2,3,4];

const newNumbers = numbers.reduce((acc, acv)=>{return acc + acv;})

console.log(numbers);
console.log(newNumbers);
```

acc – previous value
acv _ current value

```
▶ (4) [1, 2, 3, 4]
10
>
```

- *concat()*
  - faz a concatenação de dois arrays. Os arrays originais não são alterados.

```
const numbers =[1,2,3,4];

const addNumbers = [5,6];

console.log(numbers.concat(addNumbers));
```

```
▶ (6) [1, 2, 3, 4, 5, 6]
>
```


- *slice()*
  - retorna uma cópia parcial desde uma posição inicial até ao final (a posição final não é especificada). O array original não é alterado.

```
const numbers =[1,2,3,4];

console.log(numbers.slice(2));
```

```
▶ (2) [3, 4]
>
```

- *splice()*
  - altera o conteúdo de um array adicionando ou substituindo elementos (exemplo: remove 0 elementos a partir da posição 1, e adiciona o número 5)

```
const numbers =[1,2,3,4];
numbers.splice(1,0,5);

console.log(numbers);
```

```
▶ (5) [1, 5, 2, 3, 4]
```